

TASK 3: Restaurant Management System

- Set up backend using Django or Express.js to handle restaurant operations.
- Design database models for menu items, orders, tables, reservations, and inventory.
- Create APIs for placing orders, reserving tables, updating inventory, and viewing menu.
- Implement logic for order processing, table availability checks, and inventory auto-update.
- Optional: Add reporting features and admin access panel.

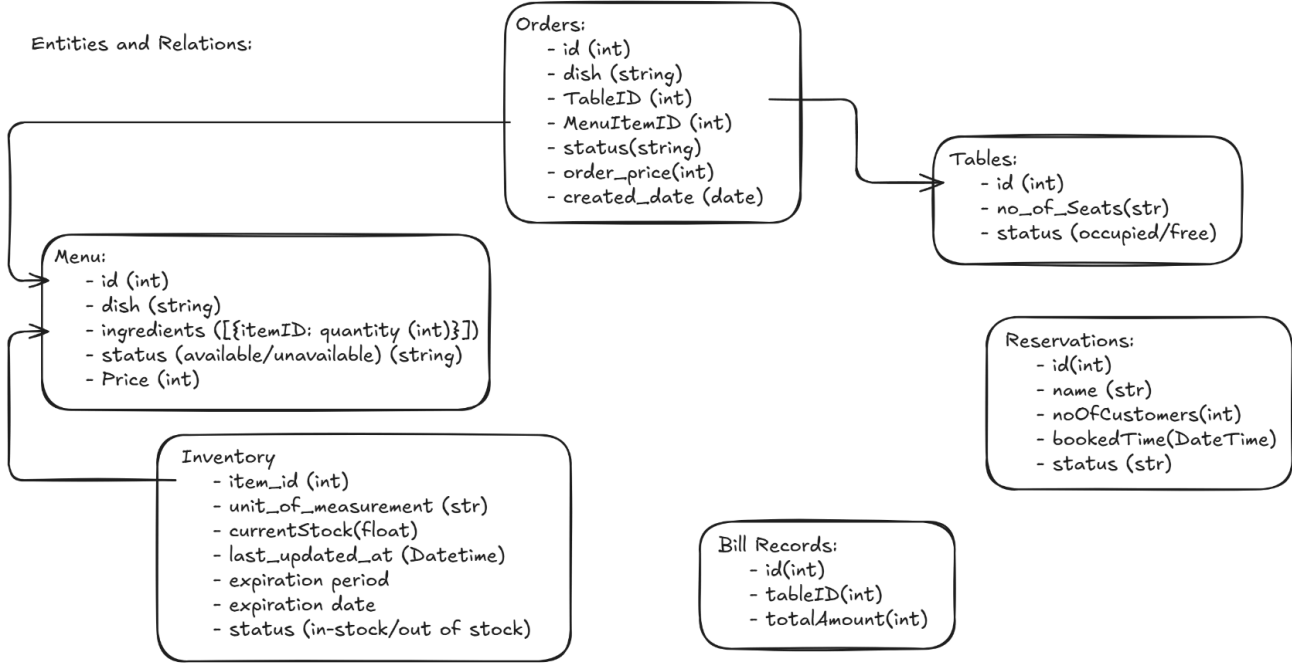
Functional Requirements:

- Table reservations
- Tables availability check
- Placing Orders
- Generate Bills per table
- Managing menu
- Managing inventory
 - Bulk insert and update inventory data
 - auto-updating inventory
- Reporting features

Assumptions:

- Inventory will be updated on a daily basis
- Bulk updating inventory will be like uploading a datafile of physical inventory check at EOD and API will handle updating all records in the Database.
- For inventory auto-update, we assume that, whenever a menu item is ordered, the ingredients required for that item will be auto subtracted from the inventory.
- For practical reasons we will allow an option for ordering of menuItems even if ingredients are not in stock from a Data perspective, cause sometimes there may be some available stock not recorded in the database.
- If any ingredient is not available for a menuItem to order, it should throw a warning saying there is a lack of ingredient.
- Reservation will be an independent to orders.
- Orders will be taken by table number.
- If an order is updated, we assume that the ingredients for previous order are already used so we don't restore item's quantity in the inventory
- Payments service will be outside of the scope for this project.

Entities and Relations:



API Endpoints:

Reservation Related:

- POST /reserve -> reqBody (ReservationObj), response : OK, -> Creates Reservation record
- GET /reserve/{name} -> PathVar ("name"), response : ReservationObj -> returns reservation record by its name
- PUT /reserve -> reqBody (ReservationObj), response : 204 -> Updates reservation record (mostly to update status)
- GET /tables/free -> returns all tables with status = "free"
- DELETE /reserve/{id} -> deletes reservation record

Order Related:

- POST /order -> reqBody (OrdersObj), response : OK -> creates an order record with status = "active", and sets the table's status to "occupied"
- GET /order/{tableID} -> returns all order records with given tableID and status = "active"
- PUT /order -> reqBody (OrdersObj), response : OK -> updates an order.
- GET /bill/{tableID} -> creates a BillObj for Orders with given TableID.
after a successful response, the all those records will have status = "finished"
and the table's status will be set to "free"

Menu Related:

- GET /menu -> reqParam("status") Return menu items without its ingredients (a separate responseObj)(only active items by default)
- POST /menu -> Creates new menu record
- PUT /menu -> Updates existing menu record
- DELETE /menu/{id} -> Soft deletes menu record
- POST /menu/restore/{id} -> Restores menu record

Inventory Related:

- POST /inventory/bulk -> bulk insert &/ update inventory records
- GET /inventory -> return all inventory records
- PUT /inventory -> updates a single inventory record
- DELETE /inventory -> deleted an inventory record

Tables Related:

- POST /tables -> create new table record
- GET /tables -> get all table records (order by status)
- PUT /tables -> updates a single table record
- DELETE /tables/{id} -> deleted an table record

Reporting Related ("/admin/report"):

- GET /orders/popular -> ReqParam(startDate, endDate) -> select menuItemName, Count(*) from orders where date >= startDate and date <= endDate
- GET /orders/traffic ->
- GET /inventory/usage ->
- GET /inventory -> filters(expiration date, stock, last updated at)

Reporting Features:

- Which dishes got ordered the most between two given Dates (to manage inventory and priority)
- Which section of the day got most tables reserved (to check traffic)
- Which ingredients got used the most
- list of all inventory with filters for expiration date(desc), stock(desc), last updated at(desc)

Event Flows

Reservation user event flow

- customer calls for reservation
- admin asks for 2 params : DateTime and Seat count
- admin will create a reservation record for that time and seat count status to active
- after customer is arrived, admin will call an endpoint to de-activate reservation

Ordering user event flow:

- Customer selects a table.
- every time waiter takes order , they create OrderRecord with table_id and order_item as input.
- the table's status will be set to occupied
- Each order with assigned table_id will be active
- after ordering is finished, waiter will call /bill/{table_id}, which will only fetch the all order records with table id and status active
- after the bill endpoint is called, the status of all records of that table_id will be "inactive" and table's status to free

Inventory management workflow:

- Whenever the order create/update endpoint is called , for each menu item , it will iterate through all its ingredients list
- for each ingredient it will get the record of that item from inventory table, subtract the current count.
- then check whether that value is less than 0.
- if <0 then it will throw error saying that this item cannot be made.
- We don't save and flush the inventory records until all items are subtracted.

[Maybe add an optional "force order" flag for practical reasons which will allow negative inventory values for that order]

Bill generation workflow:

- After calling /bill endpoint, it will fetch all records of that table id
- Create an empty HashMap<Item, quantity>
- We iterate through each record. if item present-> quantity +1 else new record
- create new Instance of Bill record object
- then use this hash map and iterate through every record.
- get the price for each item ,multiply by quantity and set total to price attribute of bill-record object. add that bill record to a List<BillRecord>
- Simultaneously keep track of entire total in a var
- after all items are done. create a Bill object, set Orders (List<BillRecord>) and Total (total)
- save the Bill object in Bill Records table for maintaining history